Chapter 7

Cluster Analysis

Overview

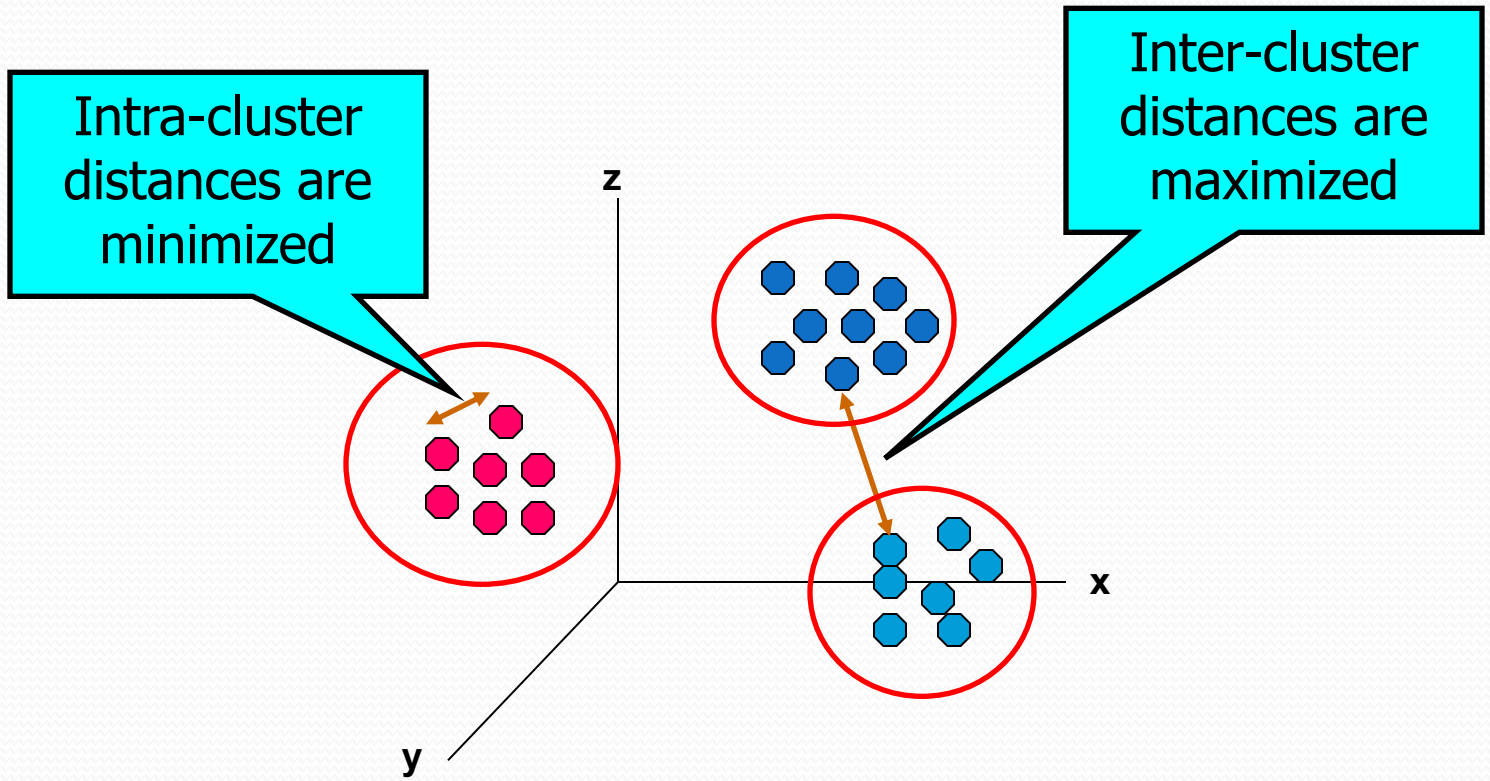
- Definition
- Partitioning methods (K-means, K-medoids)
- Hierarchical methods (HAC)
- Evaluating clustering quality

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group are similar (or related) to each other and different from (or unrelated to) objects in other groups
- "Unsupervised Learning"
  - no pre-defined class labels;
  - no examples of object groups are given.

Distance-based Cluster Analysis

- *Object similarity* is measured in terms of *distances*.



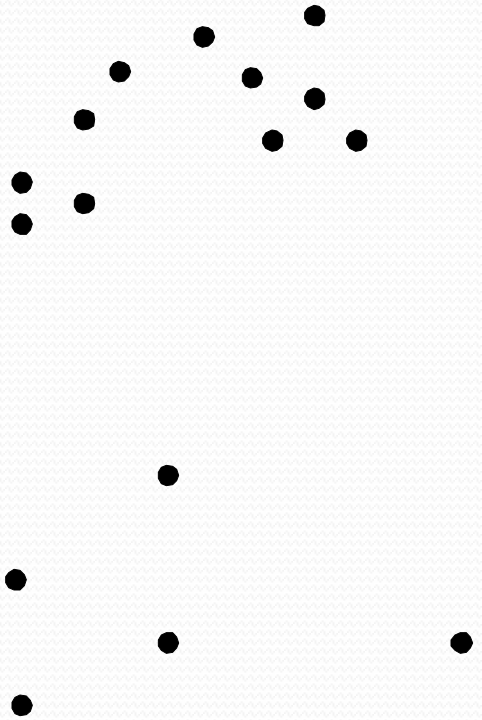
Why clustering?

- Summarization
  - Greatly reduces the size of large data set by picking a representative from each cluster and present only those representatives.
- Outlier detection
  - Objects that do not belong to any cluster are outliers.

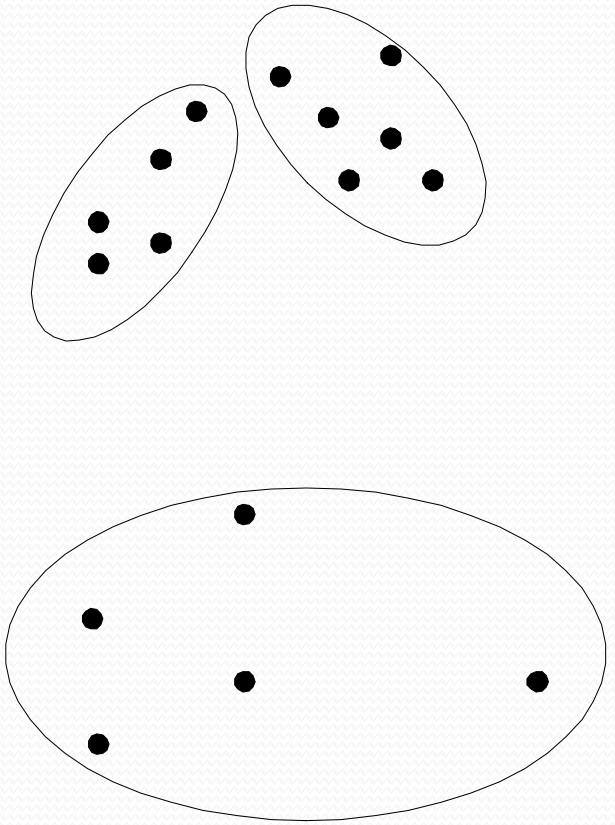
Types of Clusterings

- A *clustering* is a collection of clusters
- *Partitional Clustering*
  - A division of data objects into non-overlapping subsets (clusters) such that each data object is put into one subset (cluster)
- *Hierarchical clustering*
  - A set of nested clusters organized as a hierarchical tree

Partitional Clustering

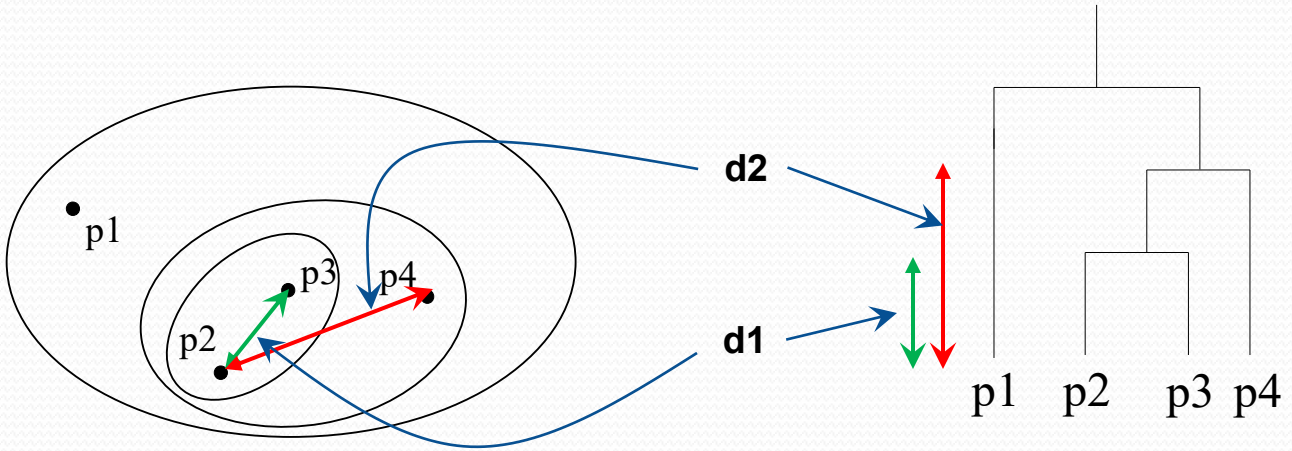


Original Points



A Partitional Clustering

Hierarchical Clustering



Hierarchical Clustering

Dendrogram

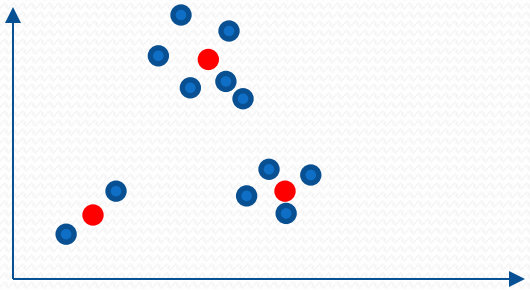
Very loose proximity requirement

Very stringent proximity requirement

Clustering Algorithms

- Distance-based clustering
  - K-means and K-medoids
  - HAC
- Density-based clustering
  - DBSCAN

K-means Clustering



- Partitional clustering
- Each cluster is associated with a centroid (center point)
- Each data object (point) x in the db is assigned to the cluster whose centroid is the closest to x
- Number of clusters, k, must be specified
- The basic algorithm is very simple

1. Select k points as the initial centroids.
2. **Repeat**
3. Form k clusters by assigning all points to their closest centroids.
4. Recompute the centroid of each cluster
5. **Until** (The centroids don't change)

Objects assignment

Centroids update

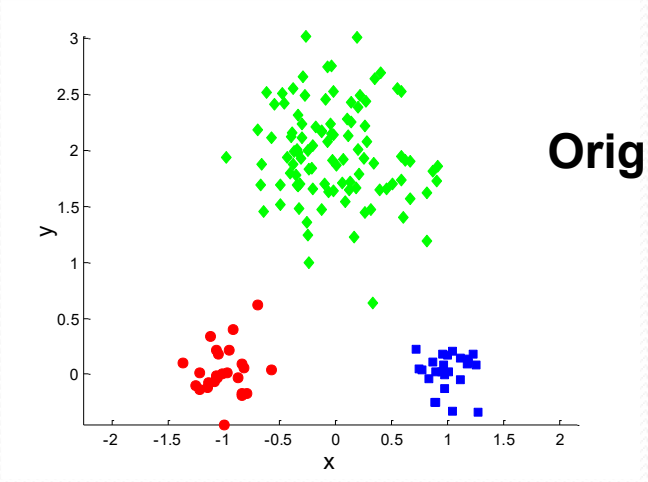
K-means Clustering – Details

- Initial centroids are often chosen randomly.
  - The clustering produced depend on the choice of the initial centroids.
- The centroid is (typically) the mean of the points in a cluster.
- "Closeness" depends on the type of data: measured by Euclidean distance, cosine similarity, etc.

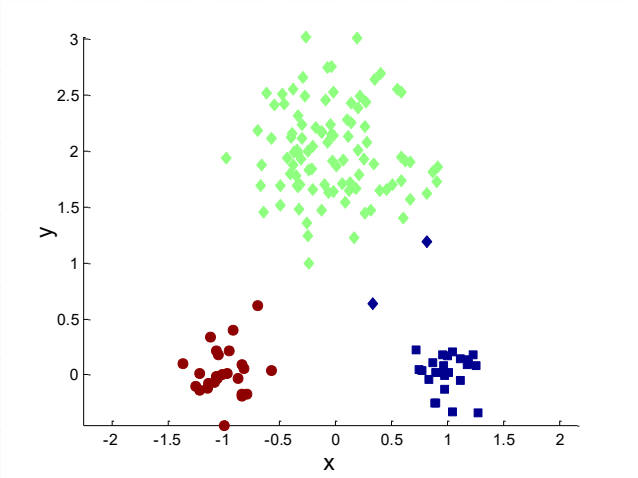
K-means Clustering – Details

- K-means will converge for common similarity measures such as Euclidean distance.
- Most of the convergence happens in the first few iterations.
  - Often the stopping condition is changed to 'no significant changes to the centroids'
- Time complexity is O(n * K * I * d)
  - n = number of points,
  - K = number of clusters,
  - I = number of iterations,
  - d = number of attributes

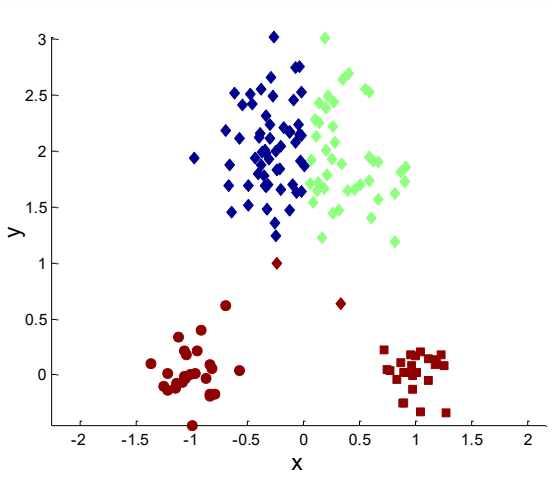
Two different K-means Clusterings (K=3)



Original Points

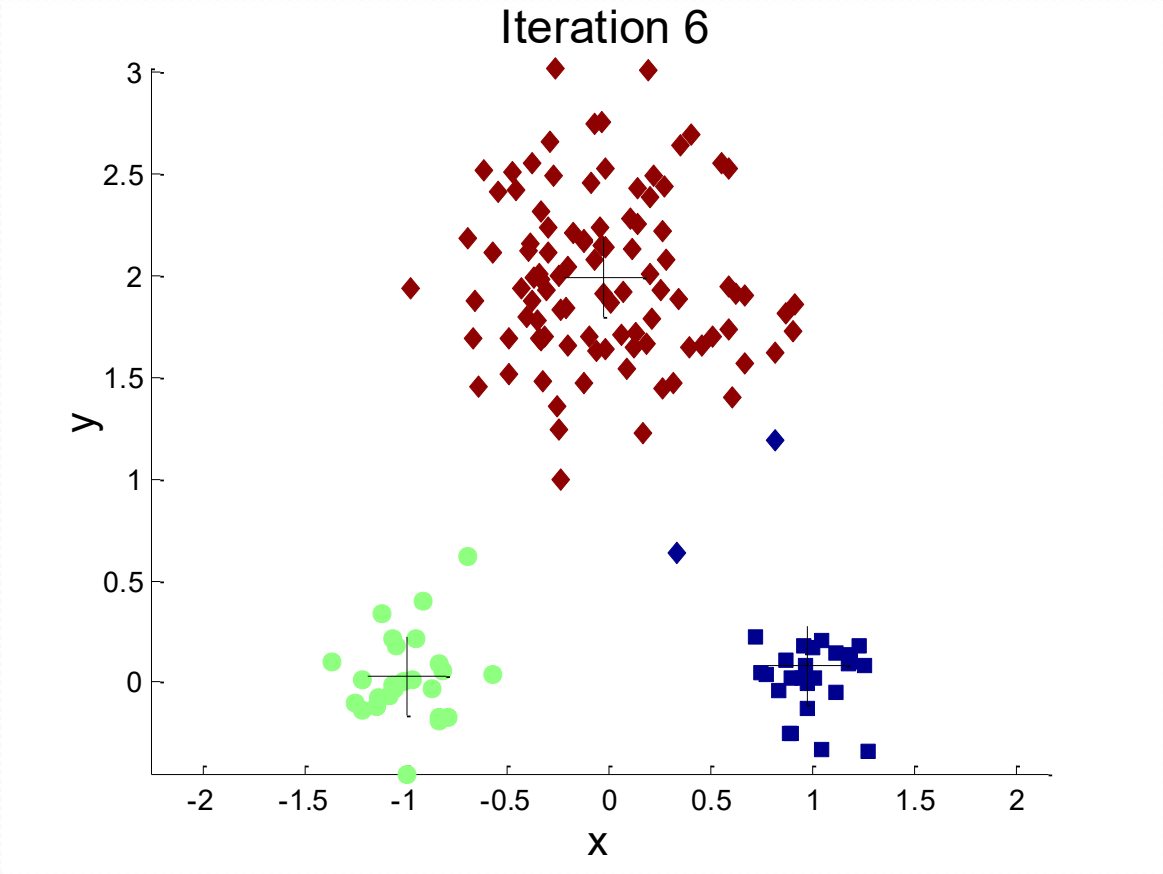


Near Optimal Clustering

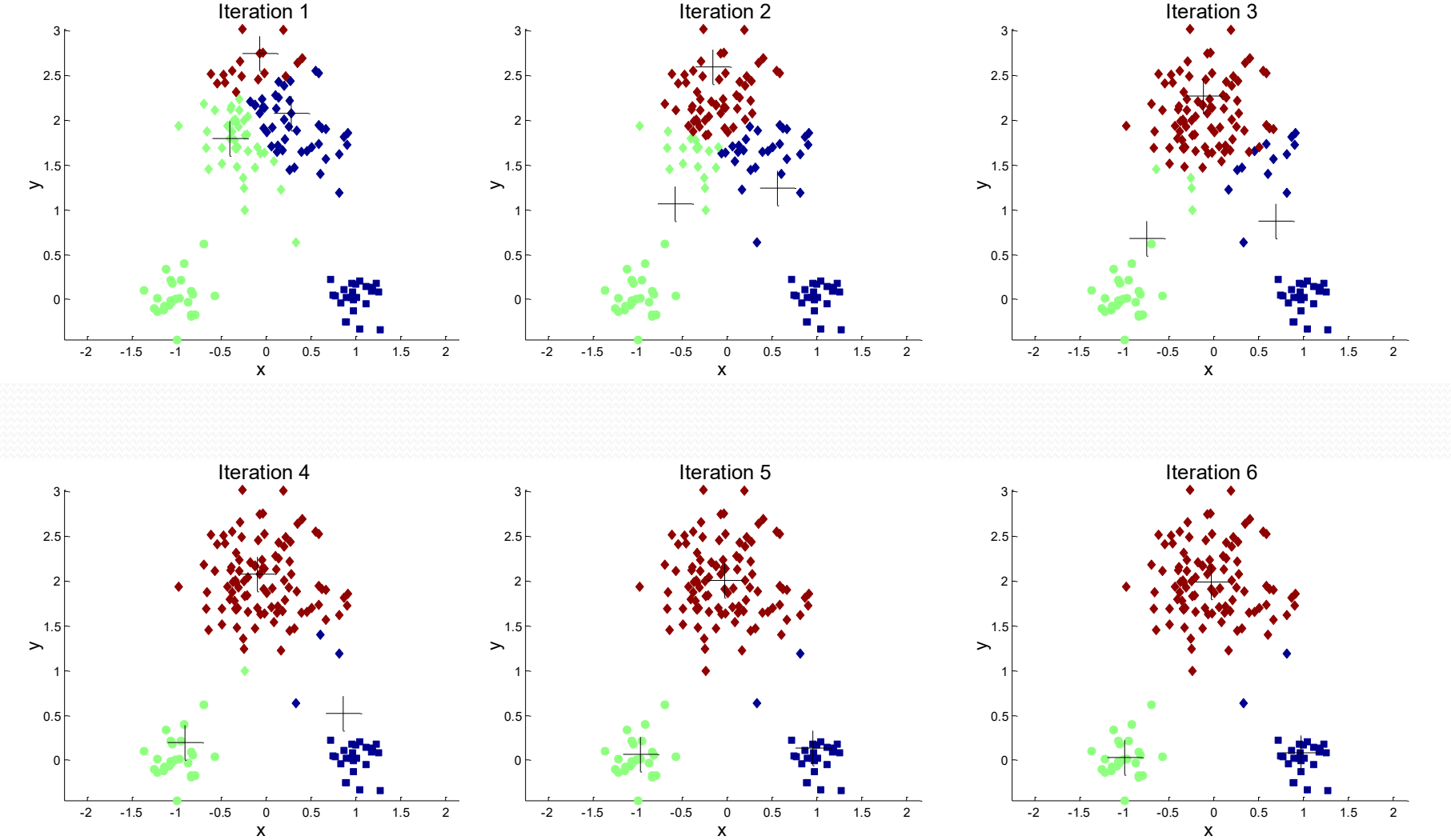


Sub-optimal Clustering

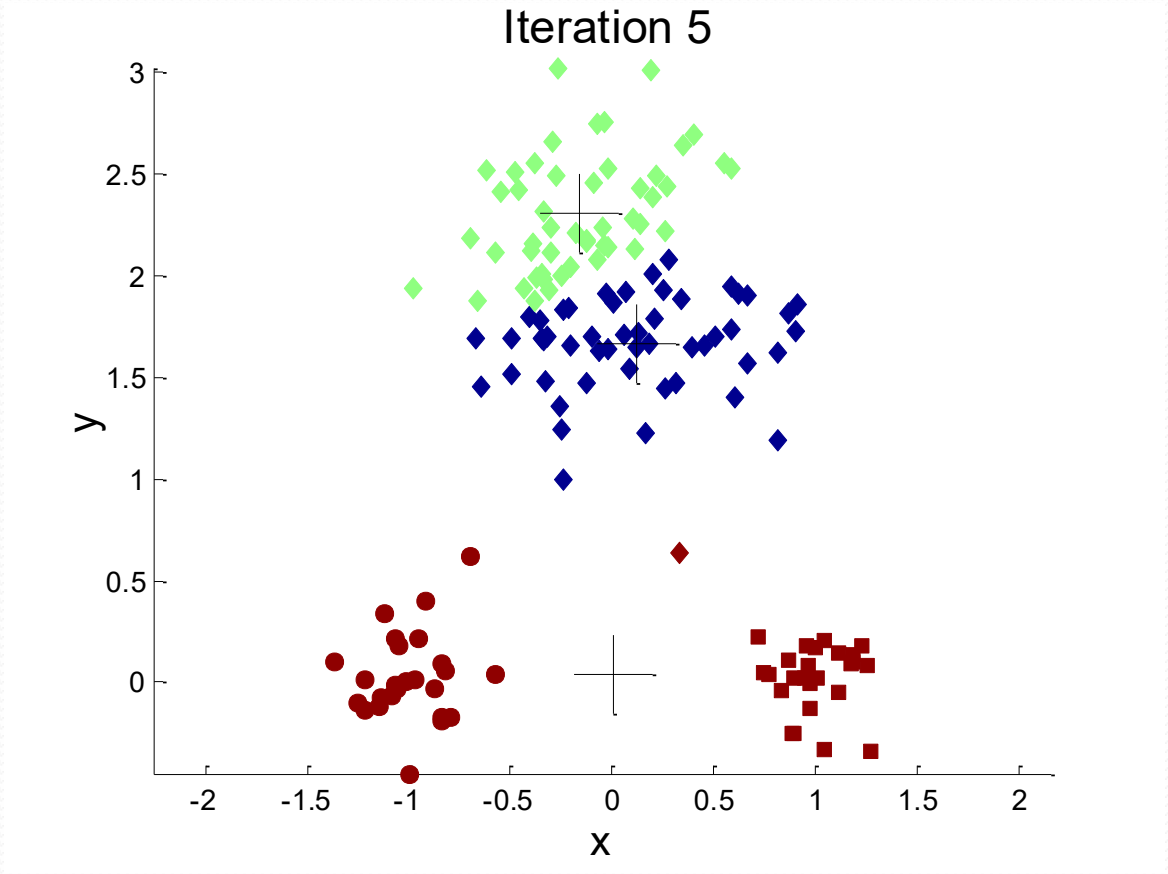
Importance of Choosing Initial Centroids



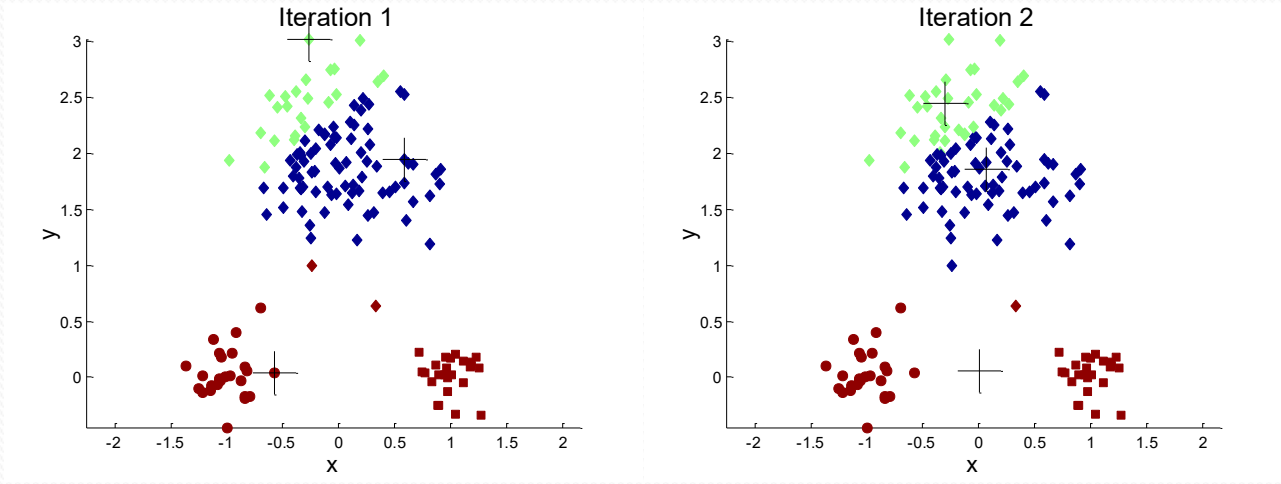
Importance of Choosing Initial Centroids

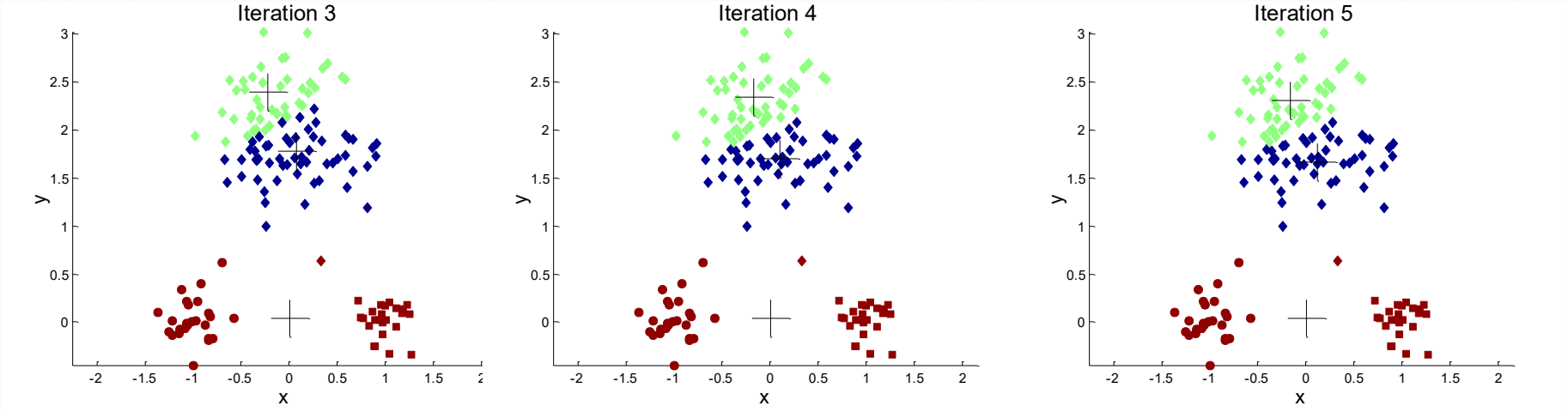


Importance of Choosing Initial Centroids ...



Importance of Choosing Initial Centroids ...





Evaluating K-means Clusters

- Most common measure is *Sum of Squared Error* (SSE)
  - For each point, the error is the distance to its cluster representative
  - To get SSE, we square these errors and sum them.

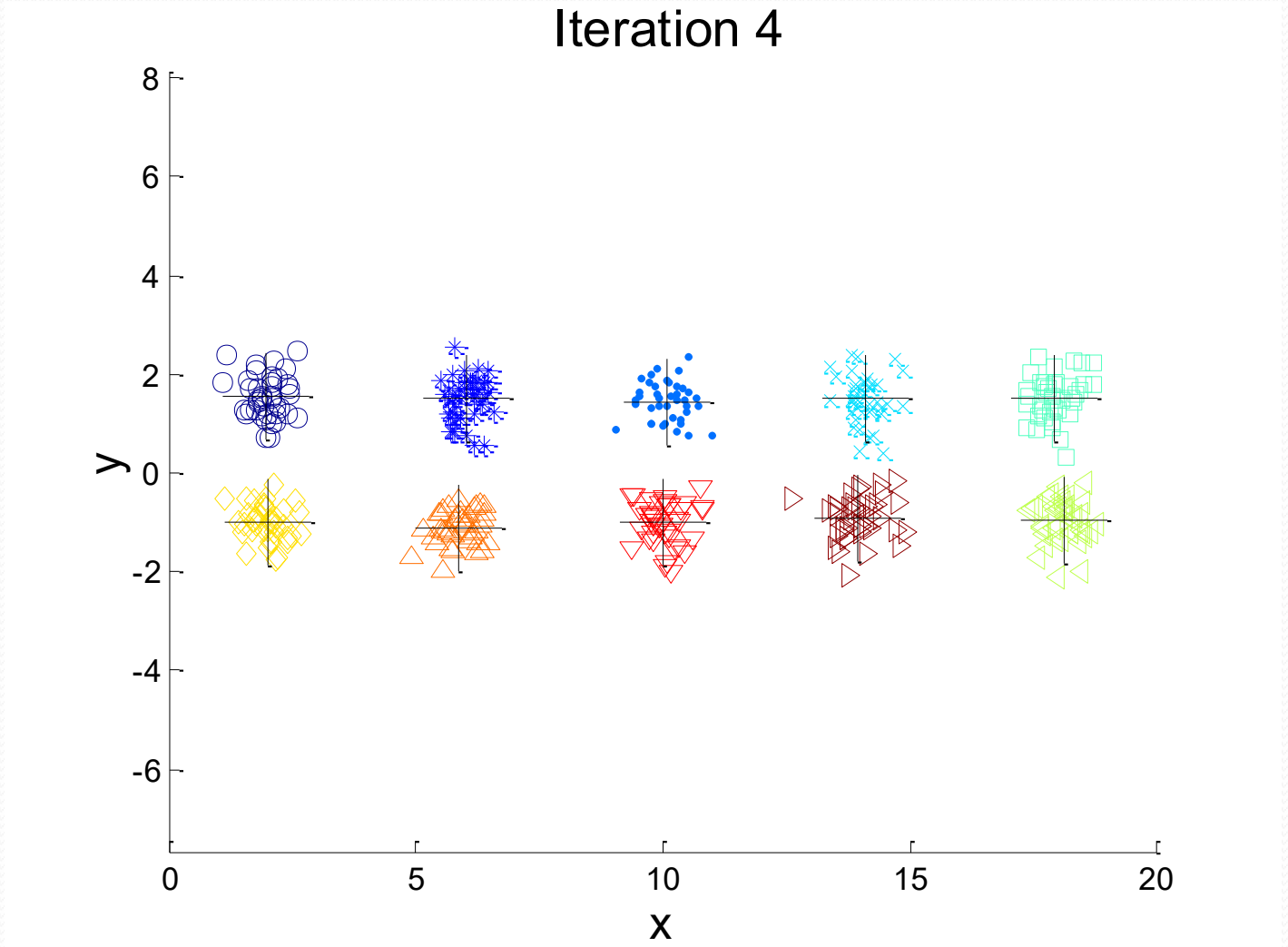
$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

SSE of Cluster C_i

SSE of the clustering

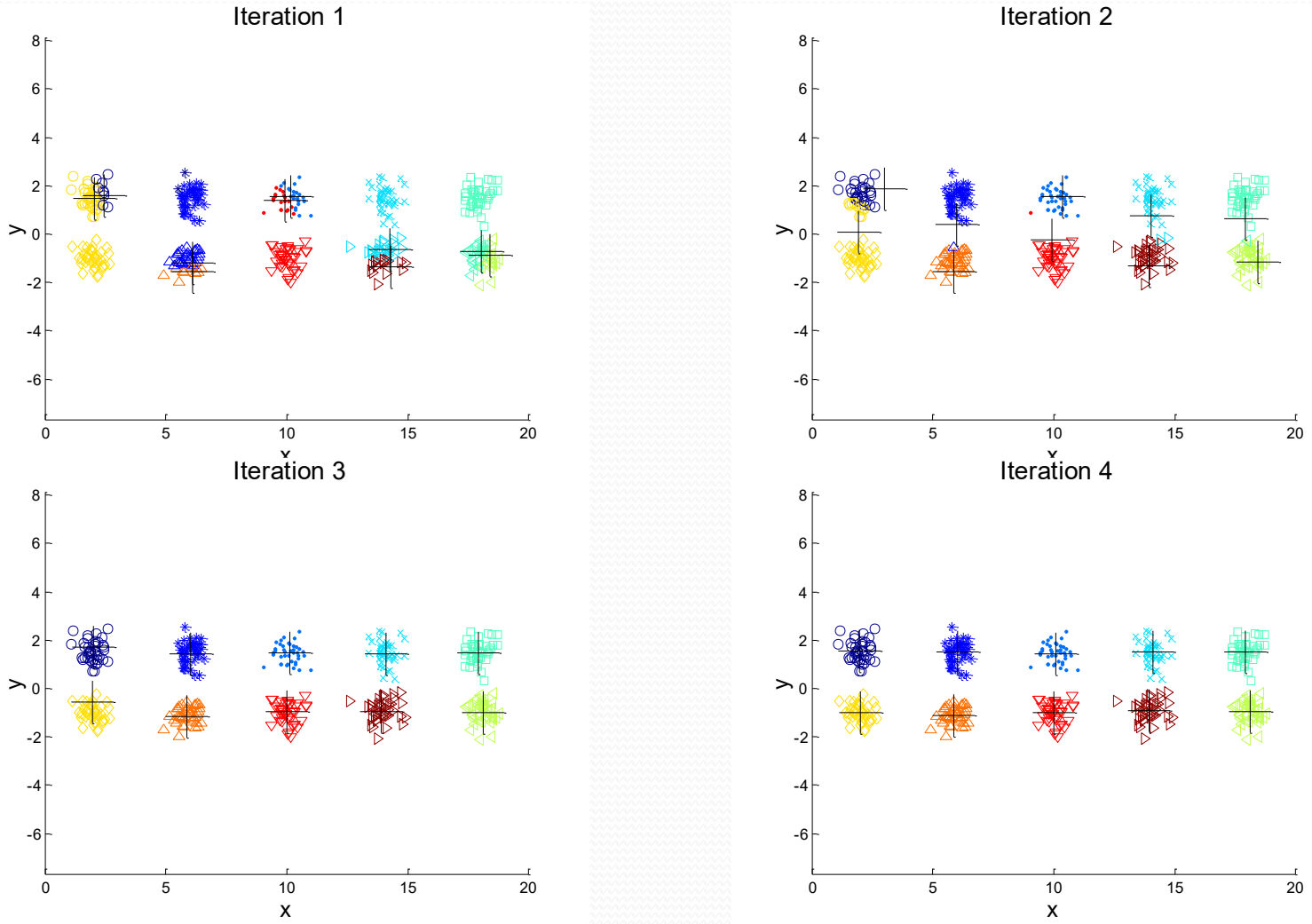
  - x is a data point in cluster C_i and m_i is the representative point for cluster C_i
    - Given a clustering, if each m_i is set as the center (mean) of its cluster, then SSE is minimized.
  - Given two clusterings, we choose the one with the smaller SSE

10 Clusters Example



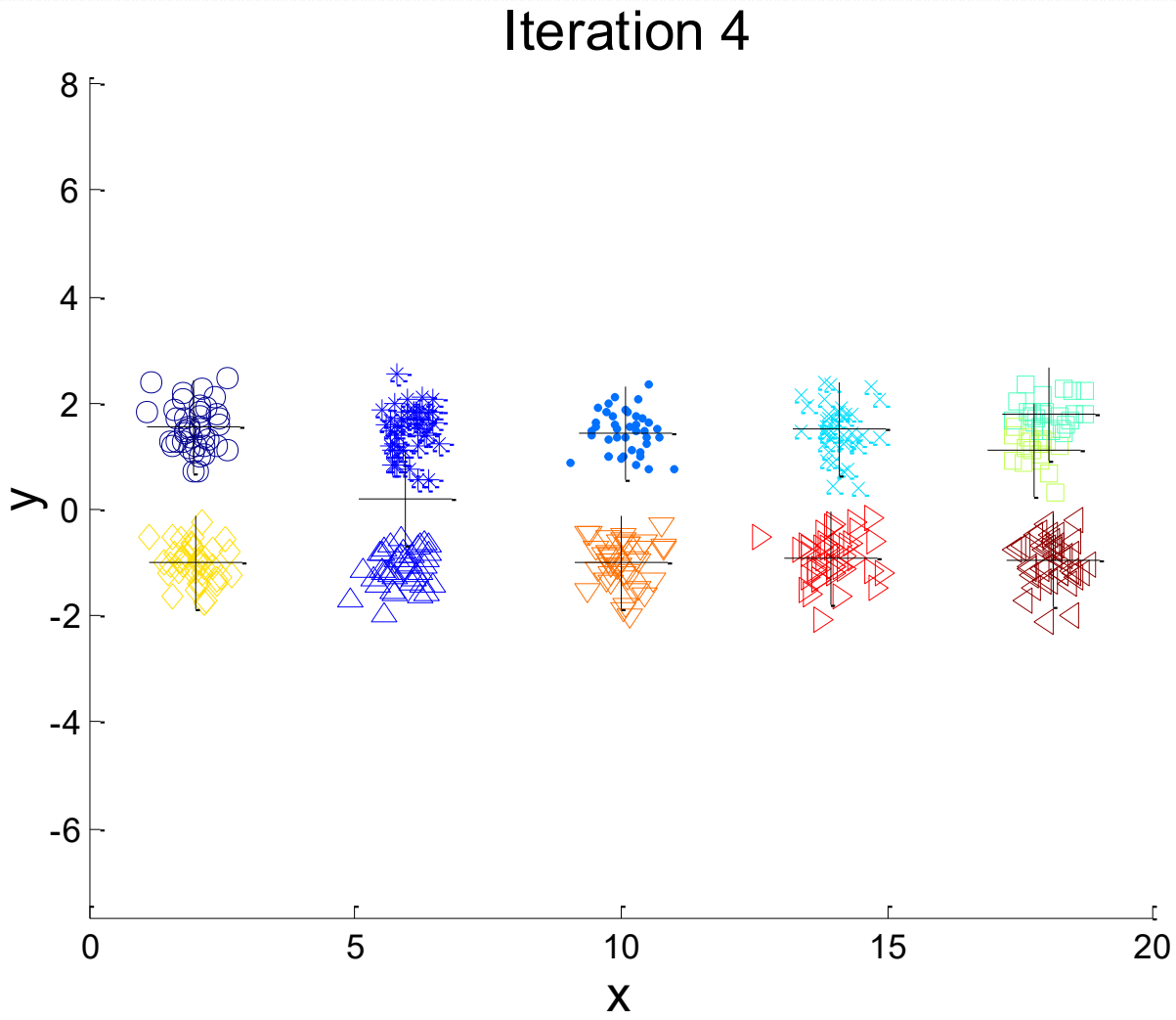
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



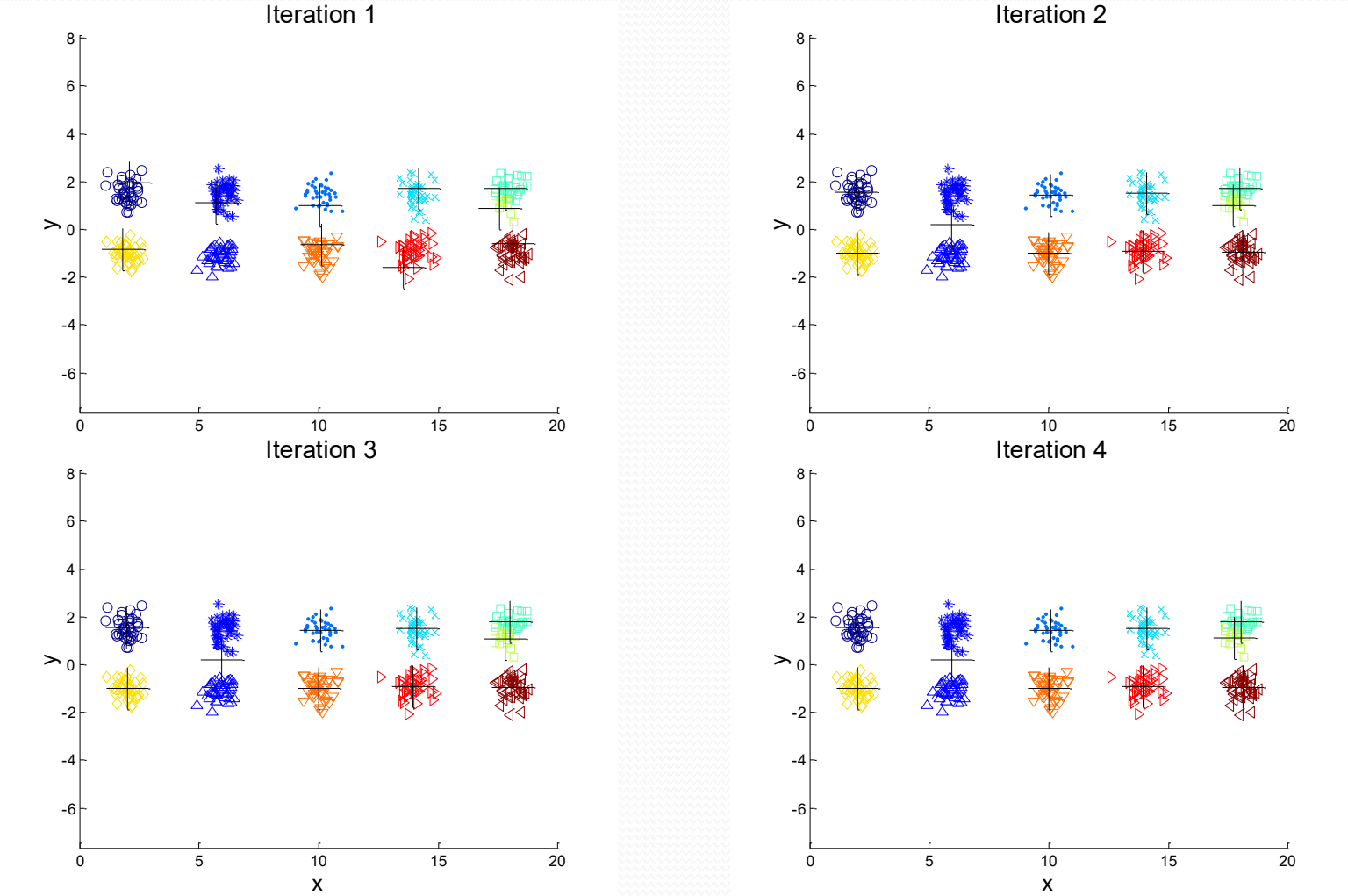
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one.

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one.

Solutions to Initial Centroids Problem

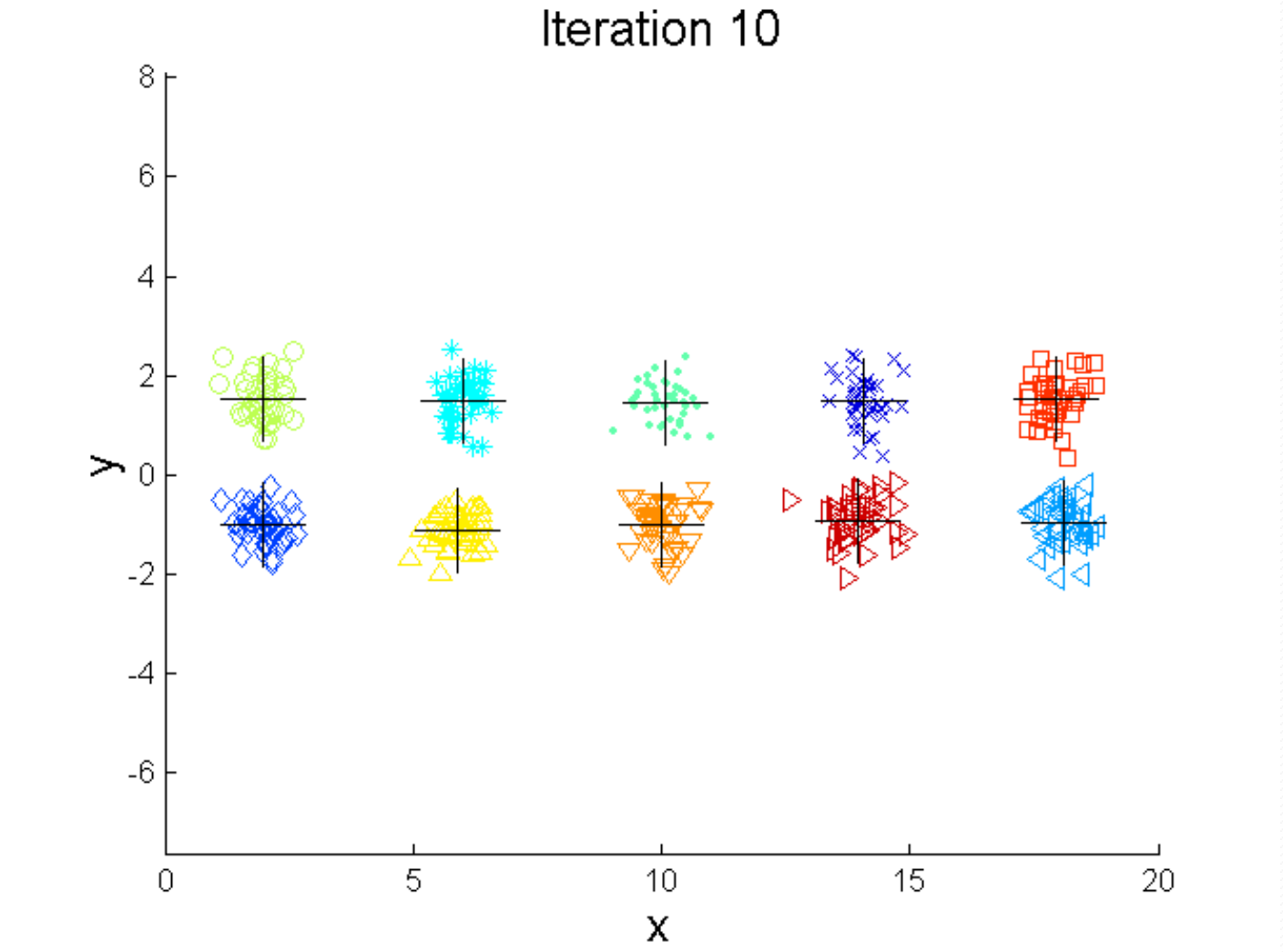
- Multiple runs
  - Pick the clustering with the best (smallest) SSE.
- Select most widely separated objects as initial centroids
- Bisecting K-means

Bisecting K-means

- Bisecting K-means algorithm
  - Variant of K-means
  - Each time pick a cluster to split
    - based on size, SSE, or both

- 1: Initialize the list of clusters to contain one cluster with all points.
  2: repeat
  3: Pick a cluster from the list of clusters.
  4: {Perform several "trial" bisections of the chosen cluster.}
  5: for i = 1 to number of trials do
  6: Bisect the selected cluster using basic K(2)-means.
  7: Select the two clusters from the bisection with the lowest total SSE.
  8: Add these two clusters to the list of clusters.
  9: until the list of clusters contains K clusters.

Bisecting K-means Example



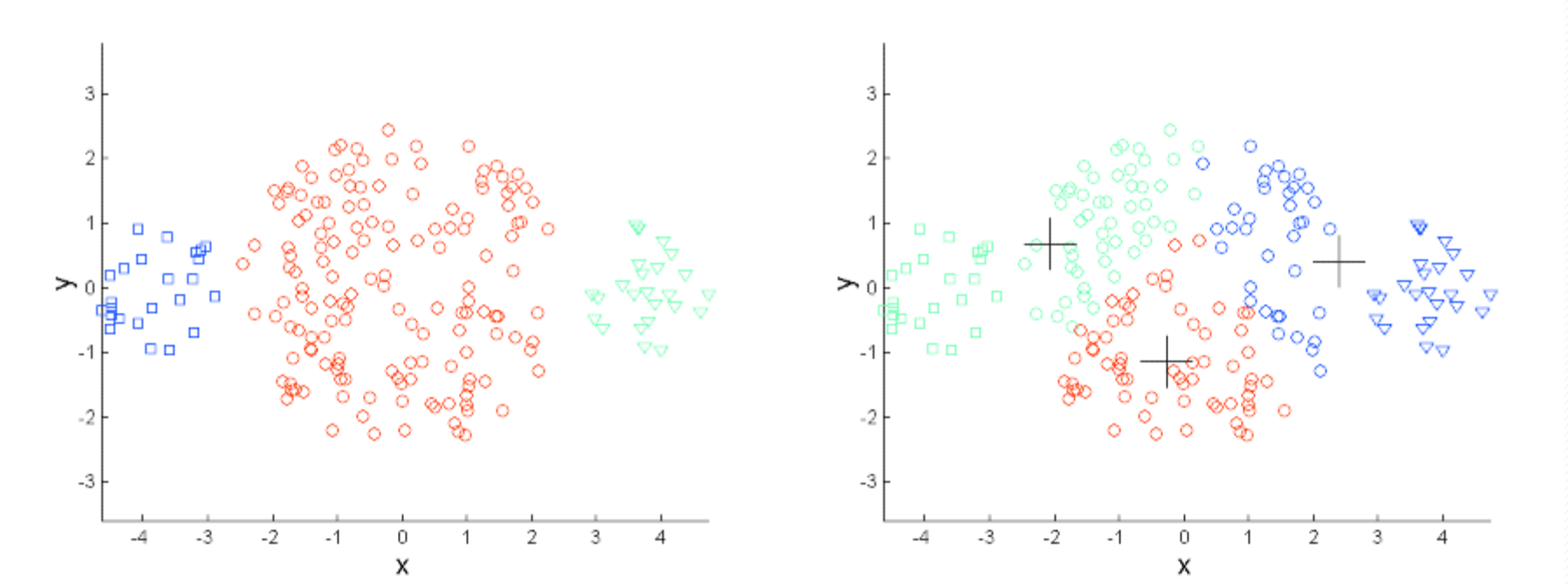
Limitations of K-means

- Not effective when clusters are
  - Of different sizes
  - Of different densities
  - Non-globular shapes

  } Multi-scale data

- K-means are susceptible to noise and outliers.

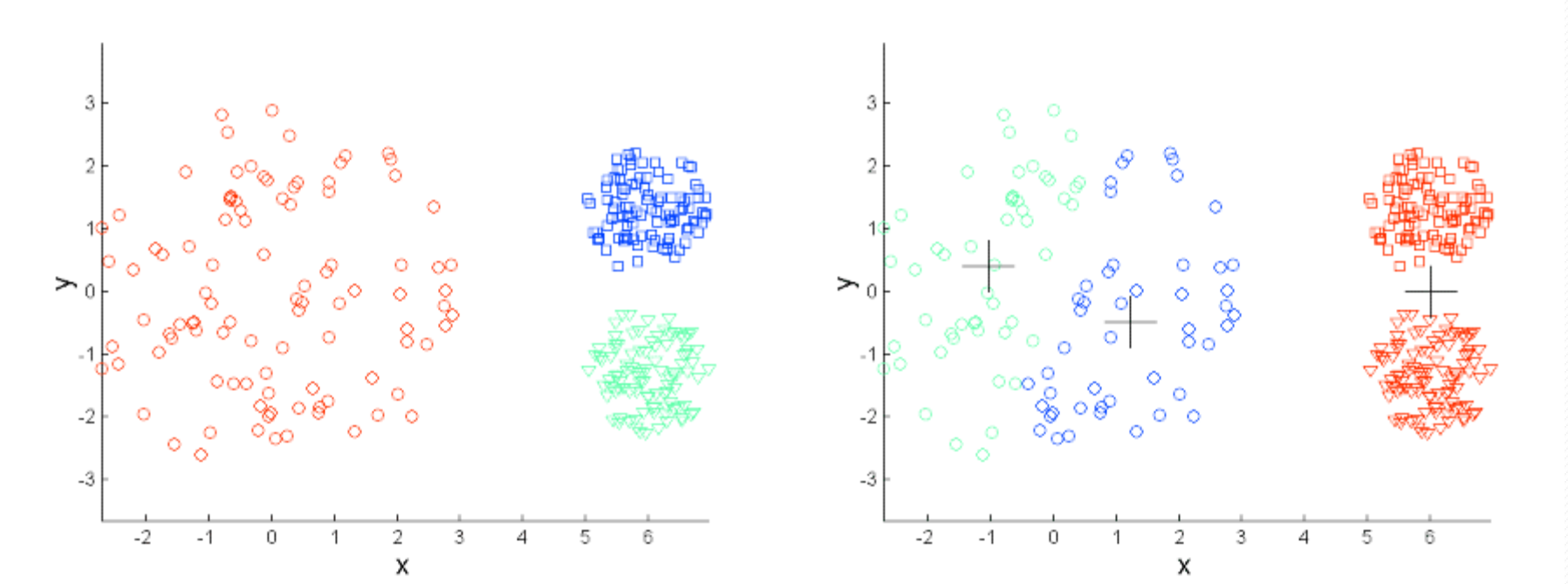
Limitations of K-means: different sizes and densities



Original Points

K-means (3 Clusters)

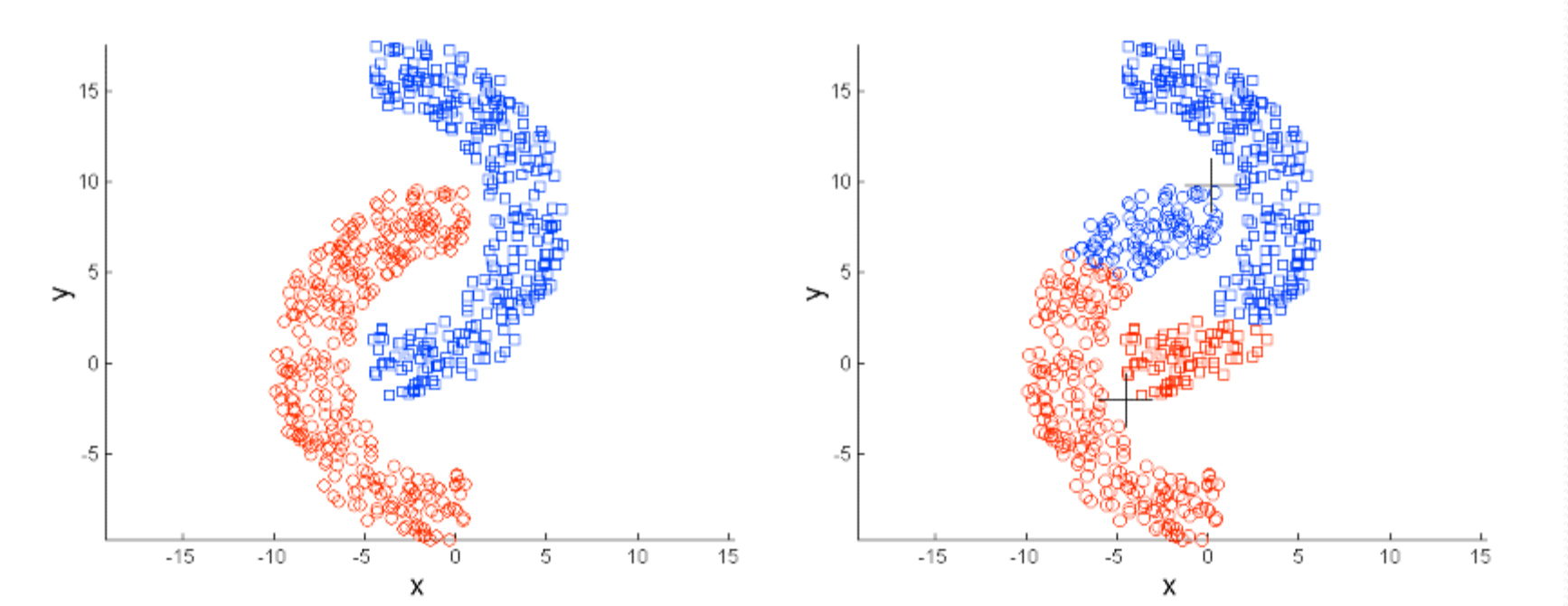
Limitations of K-means: Different sizes and densities



Original Points

K-means (3 Clusters)

Limitations of K-means: Non-globular Shapes



Original Points

K-means (2 Clusters)

k-medoids

- disadvantages of K-means:
  - averages might not make much sense for categorical/binary attributes
  - averages could be distorted by noise or outliers
- A medoid in a cluster *C* is a data object *x* in *C* such that the total distance from *x* to all other objects in *C* is minimum

k-medoids

Centroid: [0.325, 0275]

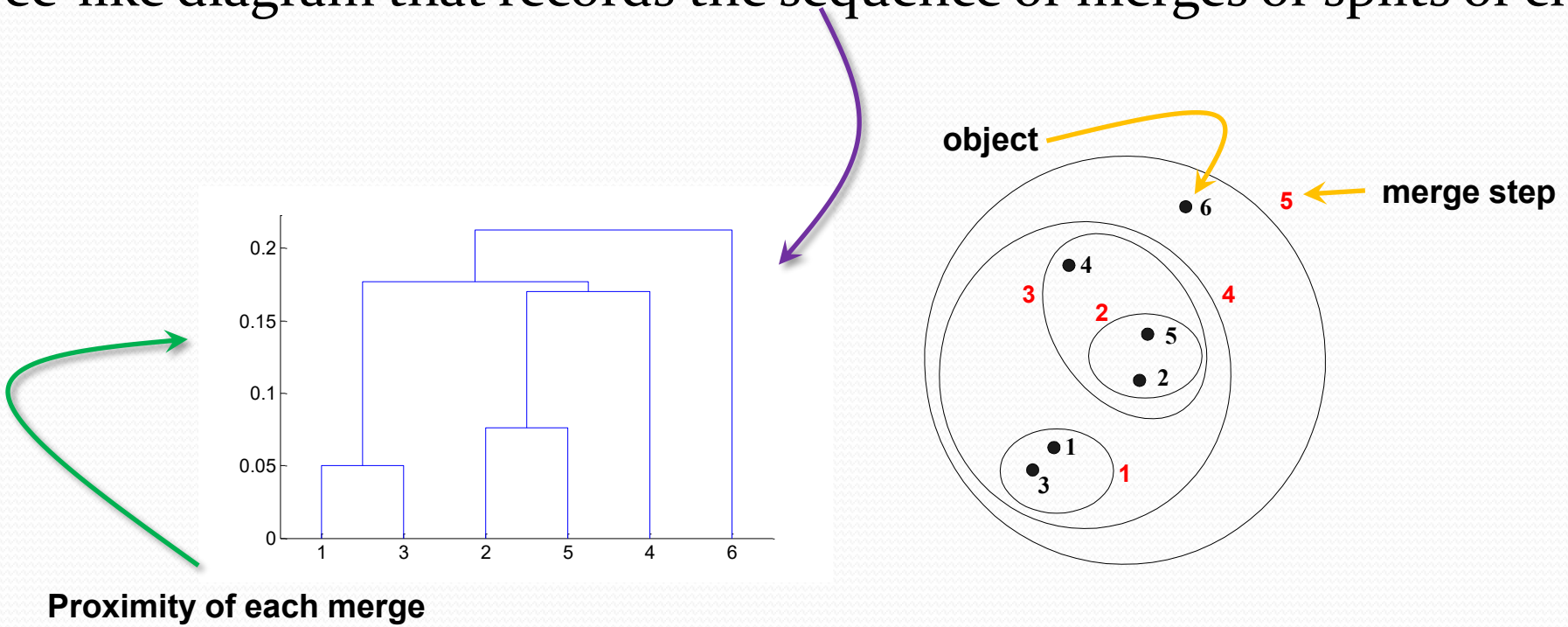
- example:
  - $x_1 = [0.3, 0.6]$, $x_2 = [0.1, 0.1]$, $x_3 = [0.5, 0.3]$, and $x_4 = [0.4, 0.1]$
  - $d(x_1,x_2) + d(x_1,x_3) + d(x_1,x_4) = 0.539 + 0.361 + 0.510 = 1.410$
  - $d(x_2,x_1) + d(x_2,x_3) + d(x_2,x_4) = 0.539 + 0.447 + 0.300 = 1.286$
  - $d(x_3,x_1) + d(x_3,x_2) + d(x_3,x_4) = 0.361 + 0.447 + 0.224 = 1.032$
  - $d(x_4,x_1) + d(x_4,x_2) + d(x_4,x_3) = 0.510 + 0.300 + 0.224 = 1.034$
  - medoid = x_3 [0.5,0.3]

k-medoids

- The K-medoids method is the same as the K-means method except that the “representative” used for a cluster is its medoid (instead of its mean)
- The K-medoids method, however, is more computationally demanding than the k-means method

Hierarchical Clustering

- Produces a set of nested clusters organized as a hierarchical tree
- Can be visualized as a *dendrogram*
  - A tree-like diagram that records the sequence of merges or splits of clusters



Strengths of Hierarchical Clustering

- Do not have to assume any specific number of clusters
  - Any desired number of clusters can be obtained by “cutting across” the dendrogram at the proper level

Hierarchical Clustering

- Two main types of hierarchical clustering
  - Agglomerative:
    - Start with the points as individual clusters
    - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left
  - Divisive:
    - Start with one, all-inclusive cluster
    - At each step, split a cluster until each cluster contains a point (or there are k clusters)
- Traditional hierarchical algorithms use a similarity or distance matrix
  - Merge or split one cluster at a time

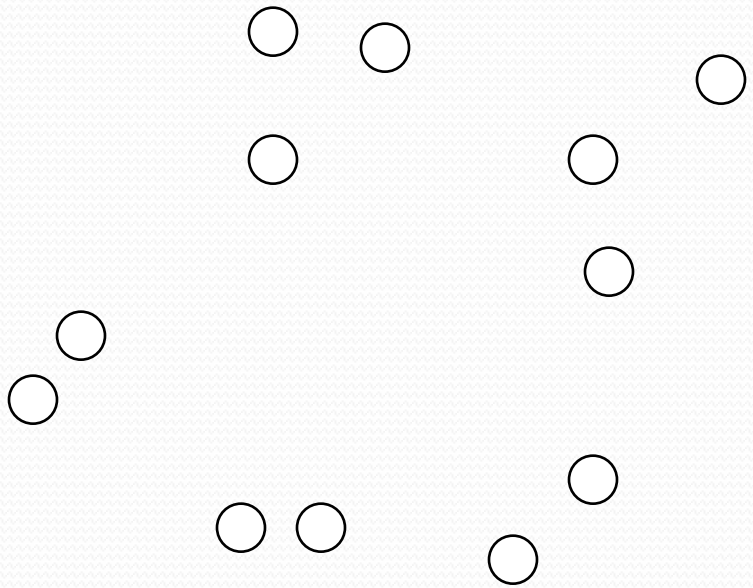
Agglomerative Clustering

- Most popular hierarchical clustering technique
- Basic algorithm is straightforward

```
Compute the proximity matrix
Let each data point be a cluster
Repeat
    Merge the two closest clusters
    Update the proximity matrix
Until only a single cluster remains
```

- Key operation is the computation of the proximity of two clusters

- Start with clusters of individual points and a proximity matrix

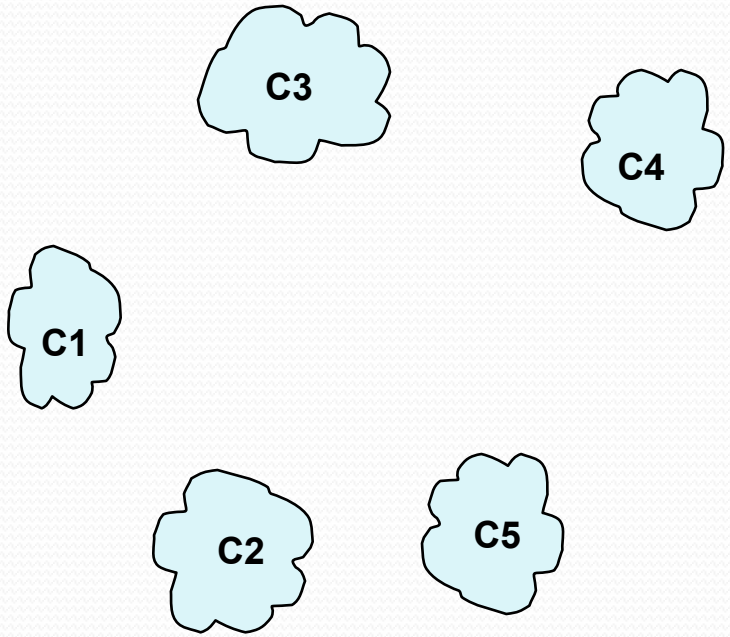


	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

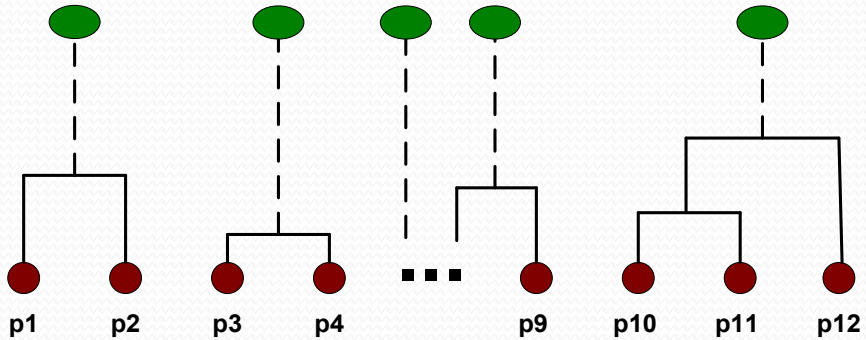
Each entry gives the proximity (either in terms of distance or similarity) between 2 clusters

- After some merging steps, we have formed some clusters

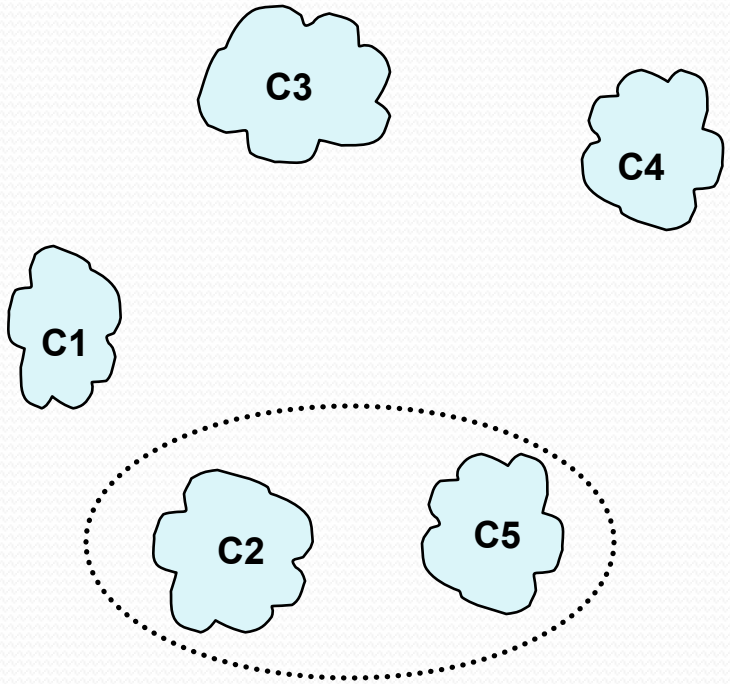


	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix

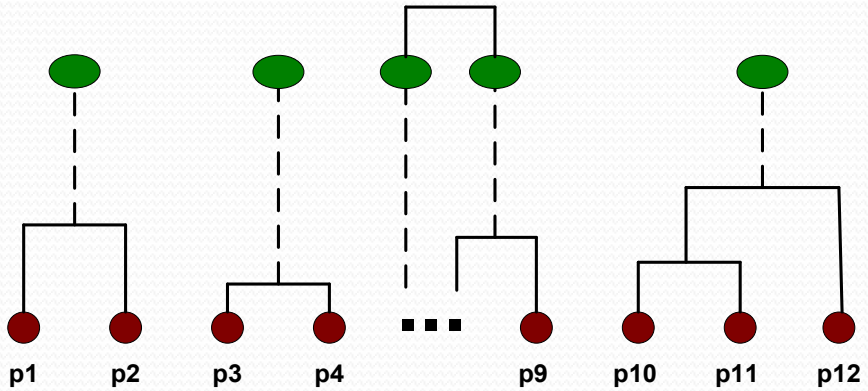


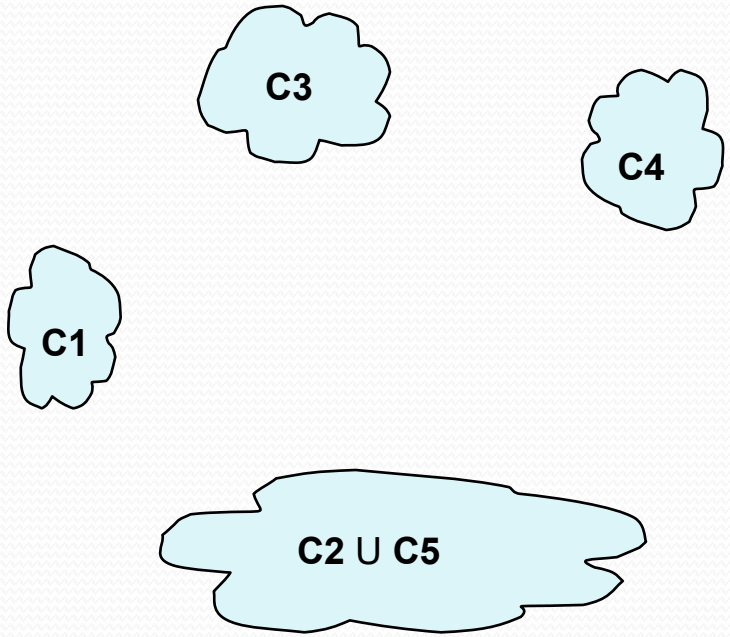
- Merge the two closest clusters (C2 and C5) and update the proximity matrix.



	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix

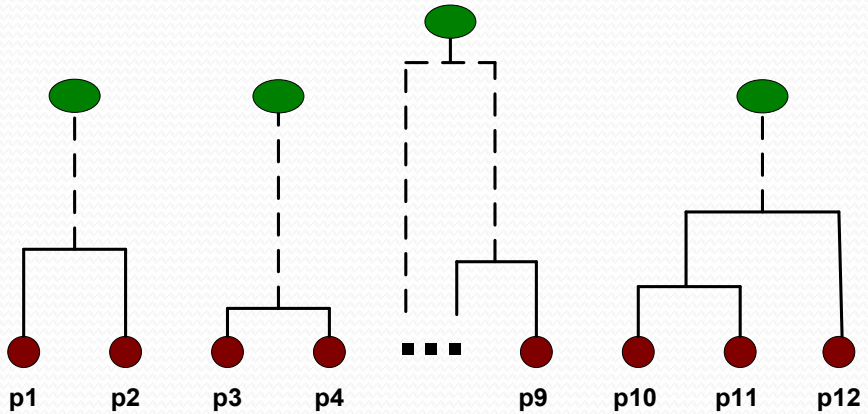




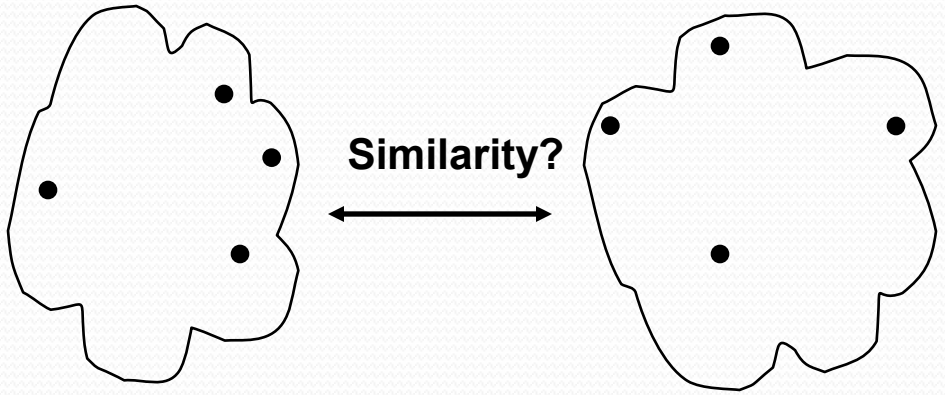
	C1	C2 U C5	C3	C4
C1		?		
C2 U C5	?	?	?	?
C3		?		
C4		?		

Compute all these proximities "?"s

Proximity Matrix



How to Define Inter-Cluster Similarity?

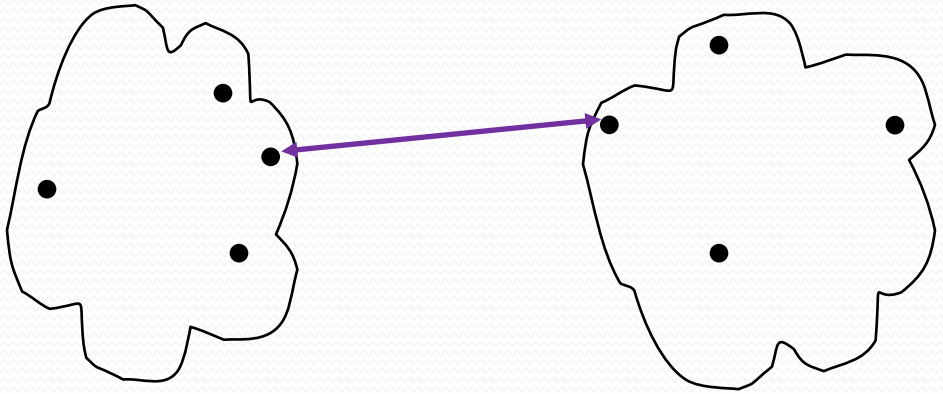


- MIN
- MAX
- Group Average
- Distance Between Centroids

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity?

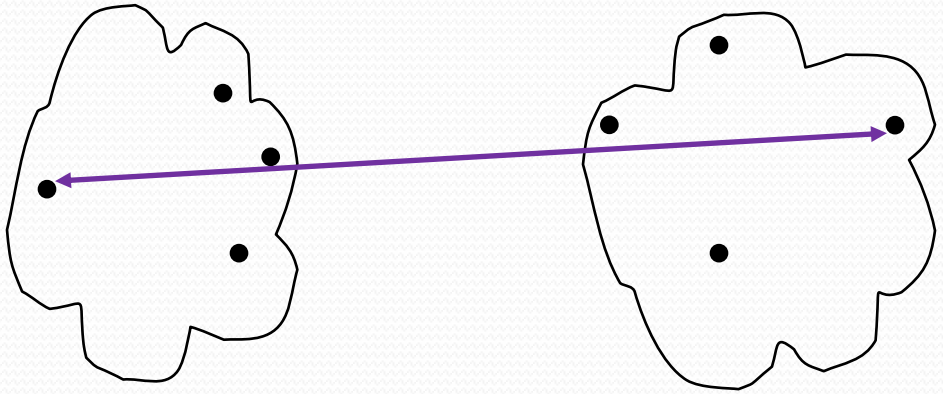


- MIN (shortest distance)
- MAX
- Group Average
- Distance Between Centroids

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity?

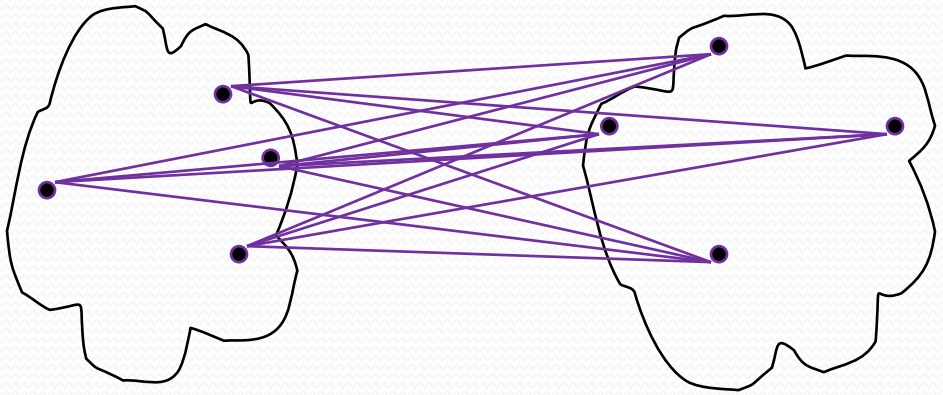


- MIN
- MAX (largest distance)
- Group Average
- Distance Between Centroids

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity?

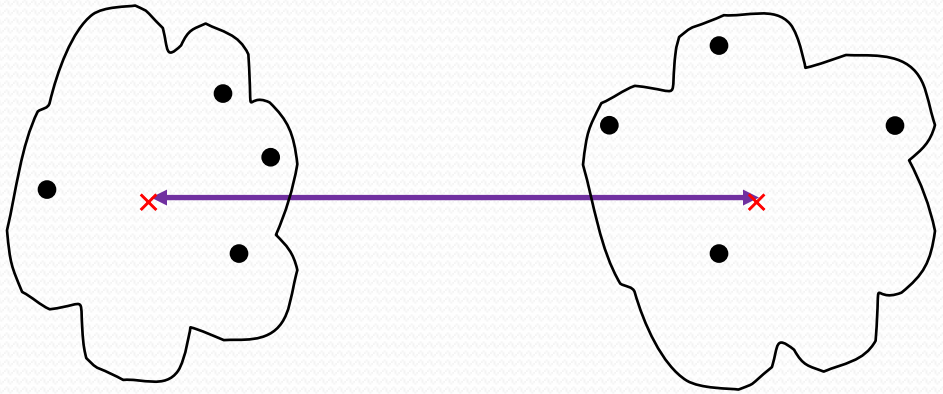


- MIN
- MAX
- Group Average (average distance)
- Distance Between Centroids

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity?



- MIN
- MAX
- Group Average
- Distance Between Centroids (distance of averages)

	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

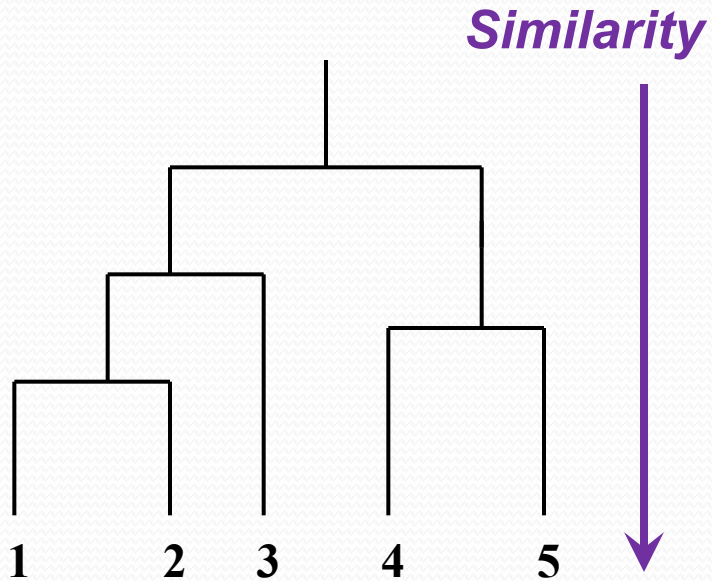
Proximity Matrix

MIN (a.k.a. Single-linkage Clustering)

- Similarity of two clusters is based on the two most similar (closest) points in the different clusters

Similarity

	I1	I2	I3	I4	I5
I1	1.00	0.90	0.10	0.65	0.20
I2	0.90	1.00	0.70	0.60	0.50
I3	0.10	0.70	1.00	0.40	0.30
I4	0.65	0.60	0.40	1.00	0.80
I5	0.20	0.50	0.30	0.80	1.00



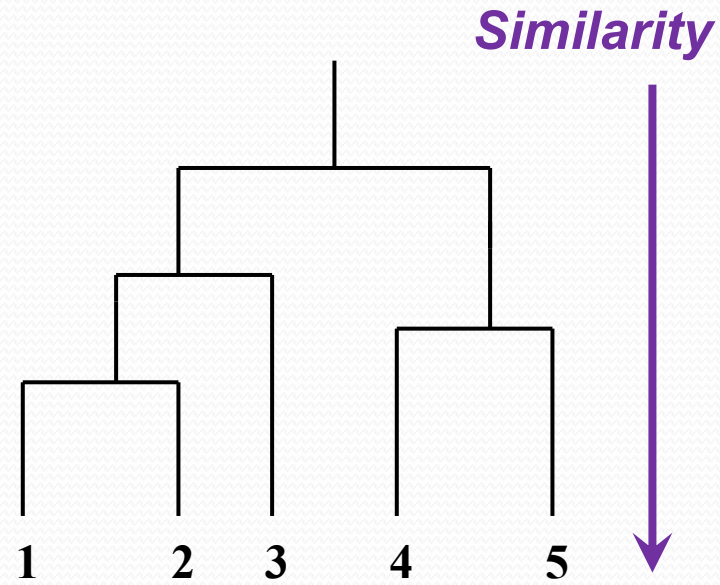
MIN (a.k.a. Single-linkage Clustering)

	I1	I2	I3	I4	I5
I1	1.00	0.90	0.10	0.65	0.20
I2	0.90	1.00	0.70	0.60	0.50
I3	0.10	0.70	1.00	0.40	0.30
I4	0.65	0.60	0.40	1.00	0.80
I5	0.20	0.50	0.30	0.80	1.00

	{1,2}	{3}	{4}	{5}
{1,2}	1.0	0.7	0.65	0.5
{3}	0.7	1.0	0.4	0.3
{4}	0.65	0.4	1.0	0.8
{5}	0.5	0.3	0.8	1.0

	{1,2}	{3}	{4,5}
{1,2}	1.0	0.7	0.65
{3}	0.7	1.0	0.4
{4,5}	0.65	0.4	1.0

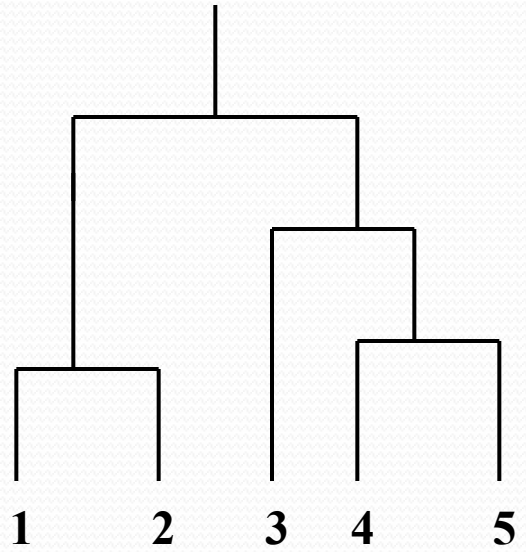
	{1,2,3}	{4,5}
{1,2,3}	1.0	0.65
{4,5}	0.65	1.0



MAX (a.k.a Complete-linkage Clustering)

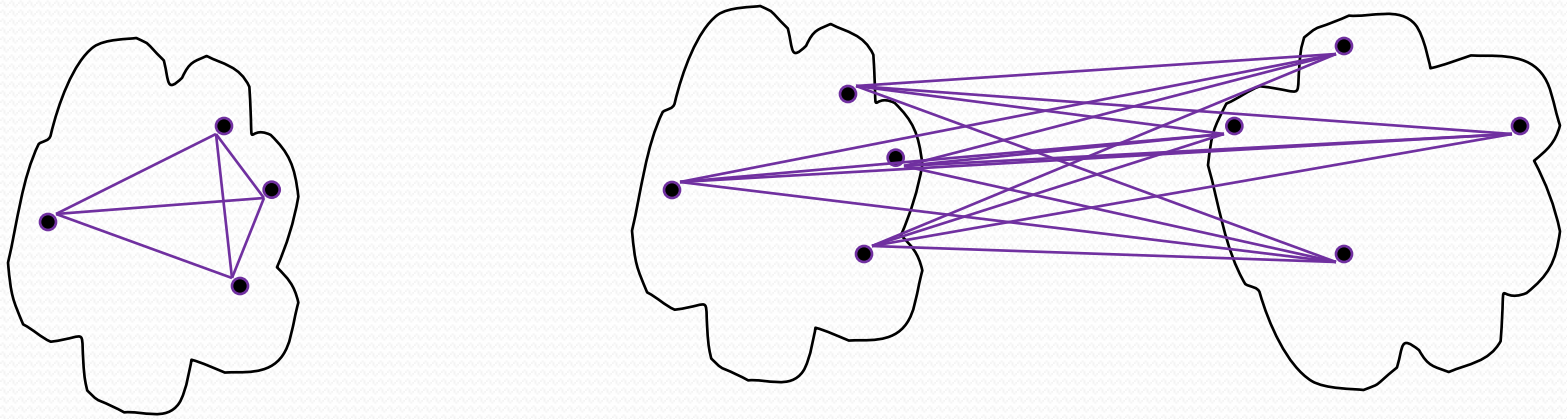
- Similarity of two clusters is based on the two least similar (most distant) points in the different clusters

	I1	I2	I3	I4	I5
I1	1.00	0.90	0.10	0.65	0.20
I2	0.90	1.00	0.70	0.60	0.50
I3	0.10	0.70	1.00	0.40	0.30
I4	0.65	0.60	0.40	1.00	0.80
I5	0.20	0.50	0.30	0.80	1.00



Measures of Clustering Quality

- SSE
- Cohesion (average intra-cluster distance) and Separation (average inter-cluster distance)

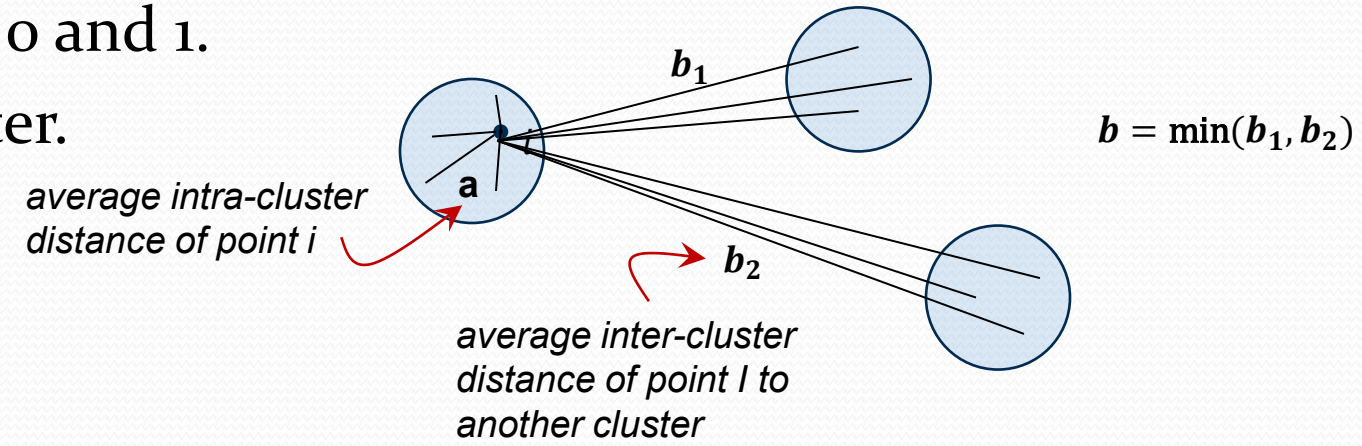


cohesion

separation

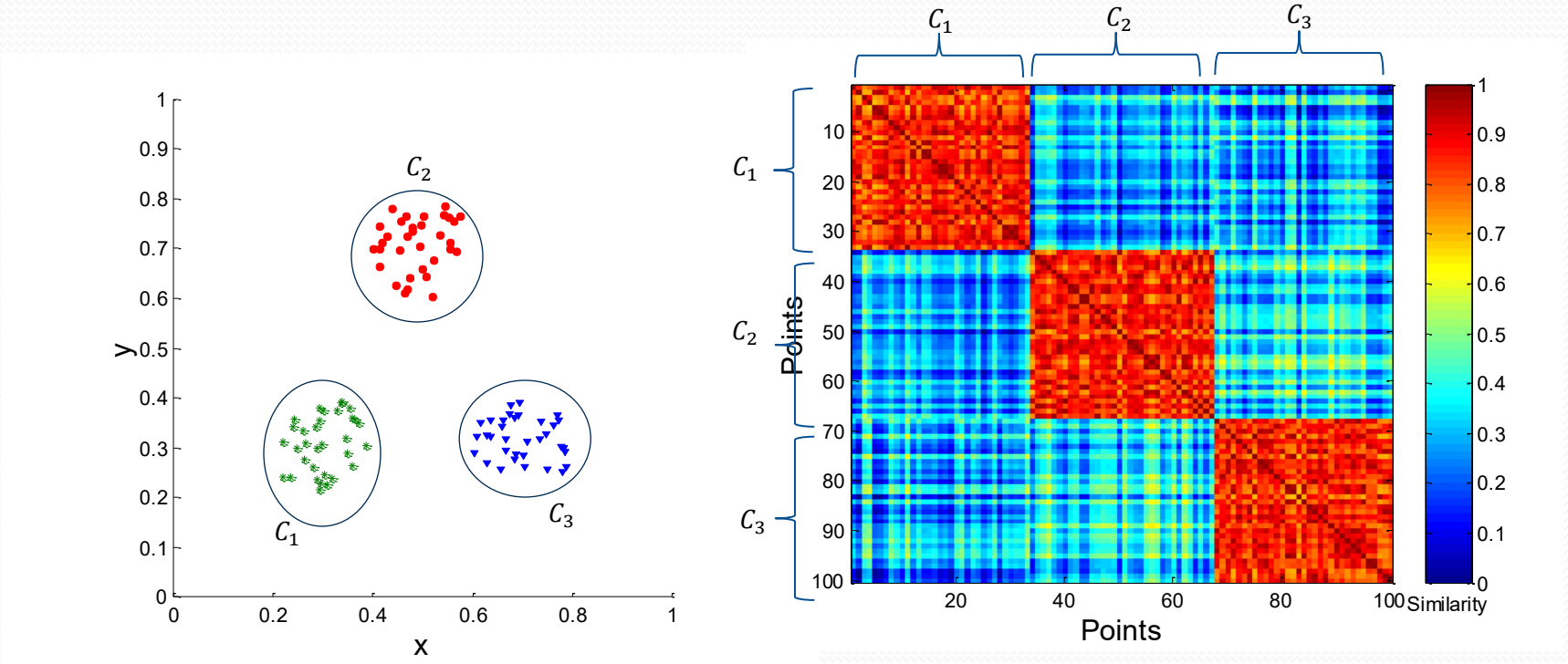
Silhouette Coefficient

- Silhouette Coefficient combines ideas of both cohesion and separation.
- For an individual point, i
  - Calculate a = average distance of i to the points in its cluster
  - Calculate b = min (average distance of i to points in another cluster)
  - The silhouette coefficient for a point is then given by
    $s = 1 - a/b$ if $a < b$, (or $s = b/a - 1$ if $a \geq b$, not the usual case)
  - Typical value between 0 and 1.
  - The closer to 1 the better.



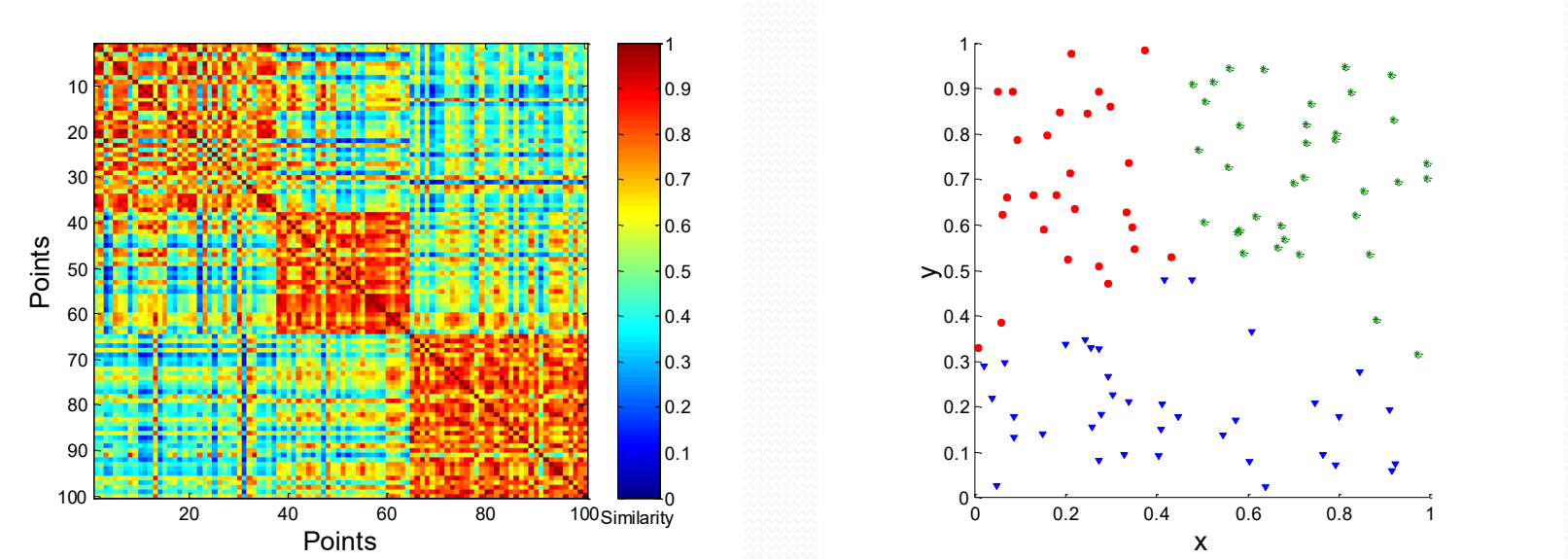
Using Similarity Matrix for Cluster Validation

- Order the similarity matrix with respect to cluster labels and inspect visually.



Using Similarity Matrix for Cluster Validation

- Clusters in random data are not so crisp



K-means